

**Mathieu MENDOUGA-NDZAMBANA, Mathis PETIT**

**Noam SLÉZACK, Ramzi MAHAMMED**

Promo 2028



## Application ArtConnect – Etape 4

## TABLE DES MATIÈRES

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Étape 1 : Comprendre ArtConnect et définir le périmètre fonctionnel .....       | 4  |
| Exploration de l'application fournie .....                                      | 4  |
| Analyse fonctionnelle .....                                                     | 4  |
| Conception statique.....                                                        | 7  |
| Étape 2 : Modélisation conceptuelle et logique de la base ArtConnect.....       | 8  |
| Identifier les entités et les relations.....                                    | 9  |
| Créer un modèle conceptuel des données (MCD) .....                              | 9  |
| Créer un modèle logique des données (MLD) .....                                 | 9  |
| Étape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées..... | 13 |
| Création et remplissage de la base .....                                        | 13 |
| Vues, index et droits d'accès .....                                             | 13 |
| Déclencheurs et programmes stockés .....                                        | 13 |
| Transactions.....                                                               | 13 |
| Étape 4 : Intégration de la base dans l'application Java ArtConnect .....       | 13 |
| Architecture de la couche de persistance .....                                  | 13 |
| Connexion à la base — DatabaseConfig et ConnectionManager .....                 | 13 |
| Implémentation des DAO JDBC .....                                               | 13 |
| Services.....                                                                   | 13 |
| Adaptation de l'interface graphique .....                                       | 13 |

Pour la bonne compréhension de ce rapport et pour permettre une lecture plus agréable, nous avons décidé de procéder par étape comme indiqué pendant les consignes du projet.

## Étape 1 : Comprendre ArtConnect et définir le périmètre fonctionnel

### Exploration de l'application fournie

L'interface est organisée autour de sept onglets principaux accessibles dans la vue principale :

- Discover
- Artists
- Artworks
- Galleries
- Exhibitions
- Workshops
- Community

À ces onglets s'ajoute, depuis l'étape 4, une fenêtre de connexion (LoginView) qui s'affiche au lancement de l'application, ainsi qu'une vue d'inscription (RegisterView) et un panneau d'administration (AdminController) accessible uniquement aux utilisateurs ayant le rôle ADMIN.

### Analyse fonctionnelle

Les fonctionnalités présentes dans l'application sont les suivantes :

- Consulter les contenus mis en avant sur la page Discover ;
- Consulter et rechercher des artistes par nom et par discipline ;
- Consulter la liste des œuvres ;
- Consulter la liste des galeries ;
- Consulter la liste des expositions ;
- Consulter la liste des workshops et s'y inscrire (utilisateur connecté) ;
- Consulter la liste des membres de la communauté ;
- Se connecter avec un compte utilisateur (login / logout) ;
- Créer un compte (inscription avec BCrypt) ;
- Gérer l'ensemble des entités via l'interface d'administration (CRUD complet réservé à l'administrateur).

#### **a) Visiteur / membre de la communauté**

Un utilisateur authentifié avec le rôle USER peut :

- Consulter les artistes, œuvres, galeries, expositions et membres ;
- Consulter la page Discover ;
- Consulter la liste des workshops ;
- S'inscrire à un workshop via le bouton dédié (la réservation est enregistrée dans la table workshop\_registration).

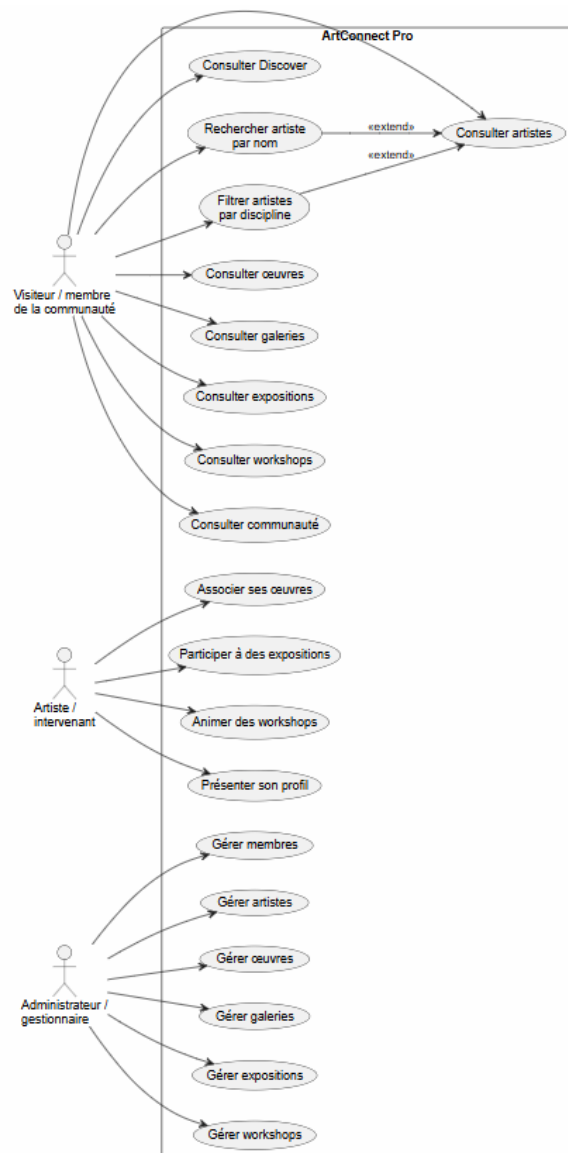
#### **b) Artiste / intervenant**

Ce rôle représente un créateur ou intervenant présent dans la base. Conceptuellement, il peut :

- Être présent sur son profil avec bio, ville, site web et réseaux sociaux ;
- Être associé à plusieurs œuvres ;
- Intervenir comme instructeur dans un ou plusieurs workshops ;
- Être lié à plusieurs disciplines artistiques via la table d'association artist\_discipline.

#### **c) Administrateur / gestionnaire de plateforme**

- L'administrateur est authentifié avec le rôle ADMIN. Via le panneau dédié (AdminController), il peut réaliser un CRUD complet sur toutes les entités : artistes, œuvres, galeries, membres, expositions et workshops. La gestion des erreurs SQL (violations de clés étrangères, doublons) est traduite en messages lisibles dans l'interface.



## Conception statique

L'architecture de l'application est organisée en packages distincts :

**model** : Artist, Artwork, ArtworkTag, Booking, CommunityMember, Discipline, Exhibition, Gallery, Review, Workshop, AppUser

**dao** (interfaces) : ArtistDao, ArtworkDao, CommunityMemberDao, ExhibitionDao, GalleryDao, WorkshopDao, UserDao

**persistence** (implémentations JDBC) : JdbcArtistDao, JdbcArtworkDao, JdbcCommunityMemberDao, JdbcExhibitionDao, JdbcGalleryDao, JdbcWorkshopDao, JdbcUserDao

**service** (interfaces) : ArtistService, ArtworkService, CommunityService, GalleryService, WorkshopService — plus AuthService et WorkshopRegistrationService

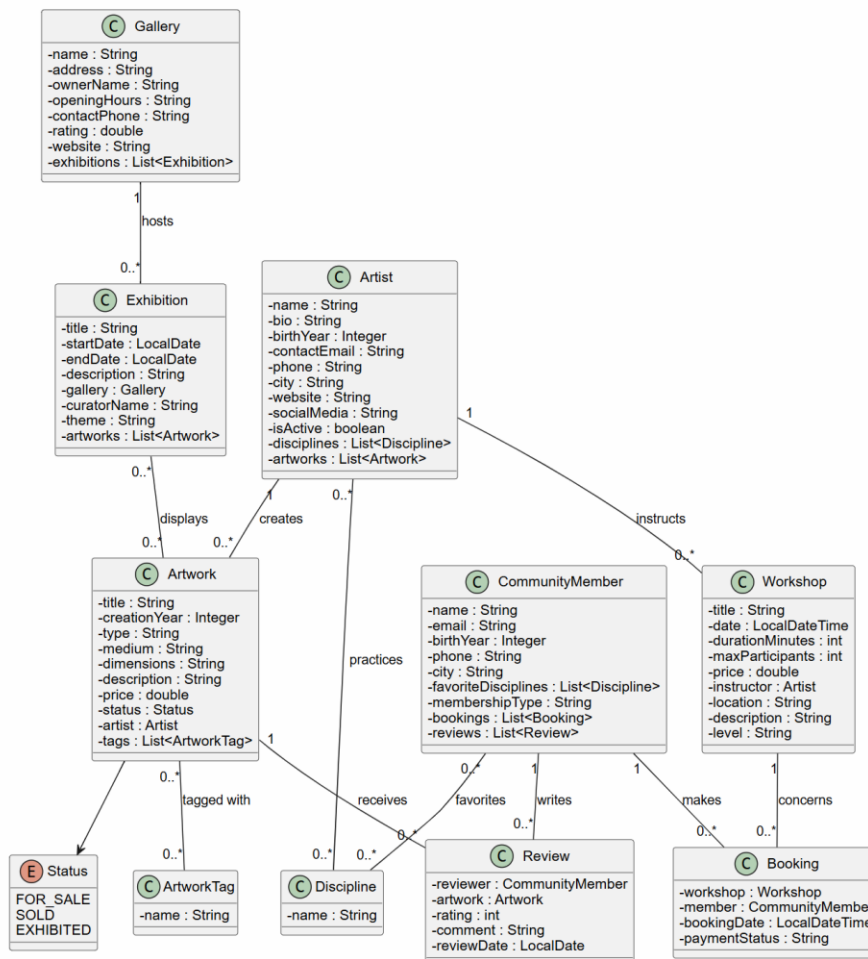
**service/impl** : JdbcArtistService, JdbcArtworkService, JdbcCommunityService, JdbcGalleryService, JdbcWorkshopService, InMemoryGalleryService (conservé pour les tests)

**ui** : MainController, ArtistController, ArtworkController, GalleryController, ExhibitionController, WorkshopController, CommunityController, DiscoverController, AdminController, LoginView, RegisterView

**config** : DatabaseConfig — URL, utilisateur et mot de passe de connexion JDBC

**util** : ConnectionManager (chargement du driver et ouverture de connexion), ServiceProvider (conteneur statique d'injection de dépendances), UserSession (session globale)

**session** : UserSession — stocke l'AppUser courant de façon statique, accessible depuis tous les controllers



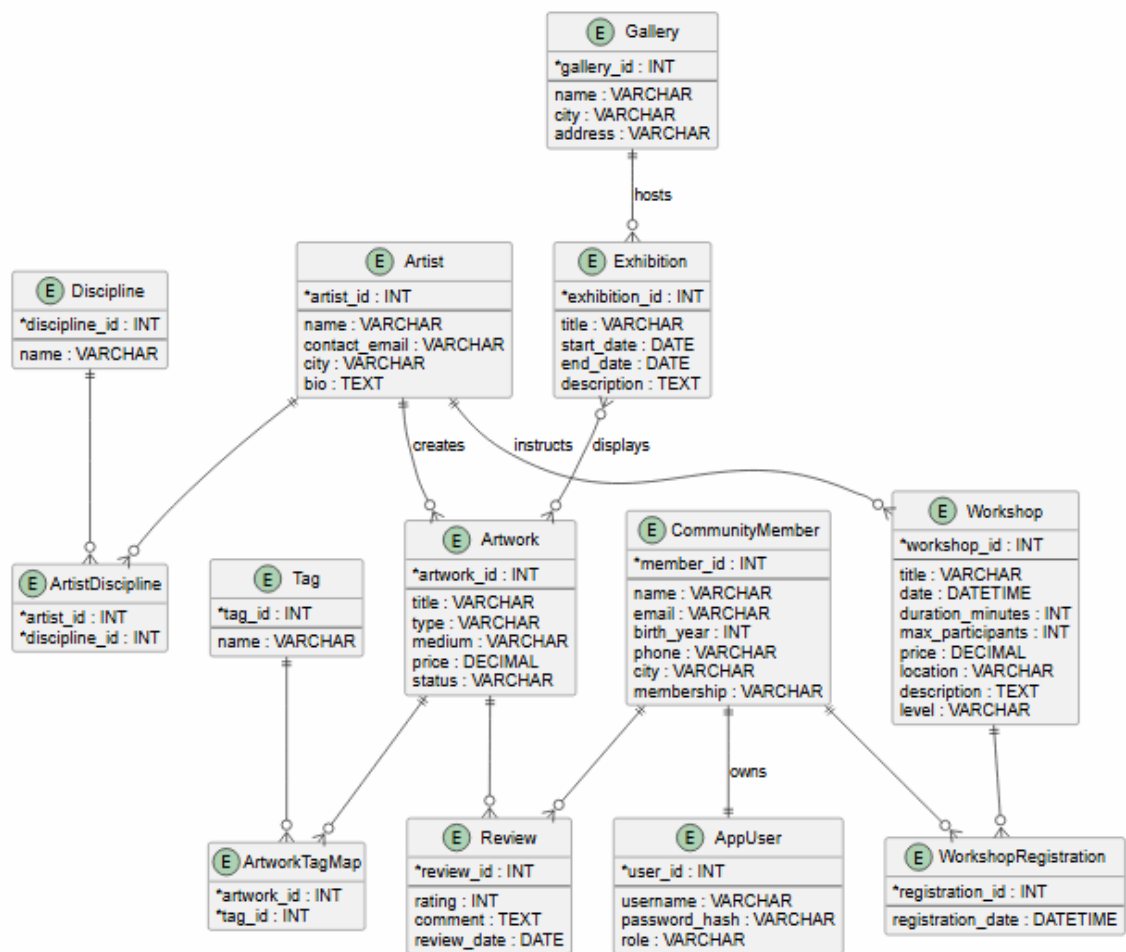
## Étape 2 : Modélisation conceptuelle et logique de la base ArtConnect

### Identifier les entités et les relations

À partir de l'interface, des besoins métier et des classes Java du modèle, nous avons identifié les entités suivantes avec leurs dépendances fonctionnelles :

- Un **artiste** possède un identifiant, un nom, une bio, une année de naissance, un email de contact unique, un téléphone, une ville, un site web, des réseaux sociaux et un statut actif/inactif.
- Une **œuvre** dépend entièrement de sa clé et appartient à un artiste (clé étrangère). Elle a un titre, une année de création, un type, un medium, des dimensions, une description, un prix et un statut.
- Une **galerie** est indépendante : elle a un nom, une adresse, un propriétaire, des horaires, un téléphone de contact, une note et un site web.
- Une **exposition** dépend d'une galerie (clé étrangère) et possède un titre, des dates, une description, un curateur et un thème.
- Un **workshop** dépend d'un artiste instructeur (clé étrangère). Il a un titre, une date, une durée, un nombre max de participants, un prix, un lieu, une description et un niveau.
- Un **membre** possède un nom, un email unique, une année de naissance, un téléphone, une ville et un type de membre.
- Un **avis (review)** lie un membre à une œuvre avec une note (1 à 5) et un commentaire.
- Une **réservation (booking)** lie un membre à un workshop avec une date de réservation et un statut de paiement.
- Une **inscription workshop (workshop\_registration)** lie un membre à un workshop de façon simplifiée, avec une contrainte d'unicité (`member_id`, `workshop_id`), utilisée par la fonctionnalité d'inscription en UI.
- Un **utilisateur applicatif (app\_user)** est lié optionnellement à un membre et possède un nom d'utilisateur unique, un mot de passe hashé BCrypt et un rôle (USER ou ADMIN).
- Les entités **discipline** et **artwork\_tag** sont des référentiels indépendants utilisés via des tables d'association.

## Créer un modèle conceptuel des données (MCD)





## Créer un modèle logique des données (MLD)

**ARTIST**( *artist\_id*, name, bio, birth\_year, *contact\_email*, phone, city, website, social\_media, is\_active )

**ARTWORK**( *artwork\_id*, title, creation\_year, type, medium, dimensions, description, price, status, #artist\_id )

→ status ∈ {FOR\_SALE, SOLD, EXHIBITED}

→ price ≥ 0

**GALLERY**( *gallery\_id*, name, address, owner\_name, opening\_hours, contact\_phone, rating, website )

→ rating ∈ [0, 5]

**EXHIBITION**( *exhibition\_id*, title, start\_date, end\_date, description, curator\_name, theme, #gallery\_id )

→ end\_date IS NULL OR end\_date ≥ start\_date

**WORKSHOP**( *workshop\_id*, title, date, duration\_minutes, max\_participants, price, *instructor\_artist\_id*, location, description, level )

→ level ∈ {Beginner, Intermediate, Advanced}

→ référence ARTIST(artist\_id) via instructor\_artist\_id

**COMMUNITY\_MEMBER**( *member\_id*, name, *email*, birth\_year, phone, city, membership\_type )

**APP\_USER**( *user\_id*, *username*, password, role, #member\_id )

→ role ∈ {USER, ADMIN}

→ member\_id nullable (ON DELETE SET NULL)

**DISCIPLINE**( *discipline\_id*, *name* )

**ARTWORK\_TAG**( *tag\_id*, *name* )

**REVIEW**( *review\_id*, rating, comment, review\_date, #reviewer\_member\_id, #artwork\_id )

→ rating ∈ [1, 5]

**BOOKING**( *booking\_id*, booking\_date, payment\_status, #workshop\_id, #member\_id )

→ payment\_status ∈ {PENDING, PAID, FAILED, CANCELLED}

**WORKSHOP\_REGISTRATION**( **registration\_id**, registration\_date, #member\_id, #workshop\_id )

→ contrainte UNIQUE(member\_id, workshop\_id)

**EXHIBITION\_ARTWORK**( #exhibition\_id, #artwork\_id )

**ARTIST\_DISCIPLINE**( #artist\_id, #discipline\_id )

**MEMBER\_FAVORITE\_DISCIPLINE**( #member\_id, #discipline\_id )

**ARTWORK\_TAG\_MAP**( #artwork\_id, #tag\_id )

La table APP\_USER et la table WORKSHOP\_REGISTRATION ont été ajoutées au schéma initial lors du développement de l'étape 4. APP\_USER supporte l'authentification BCrypt avec gestion des rôles. WORKSHOP\_REGISTRATION est distincte de BOOKING : elle est utilisée par le bouton d'inscription de l'onglet Workshops et ne stocke que la date d'inscription (sans statut de paiement), avec une contrainte d'unicité empêchant toute double inscription.

## Justification de la normalisation (3FN)

**1FN** | Tous les attributs sont atomiques. Les collections présentes dans les classes Java (liste de disciplines d'un artiste, liste d'œuvres d'une exposition, liste de tags d'une œuvre, disciplines favorites d'un membre) ont chacune été transformées en table d'association distincte : ARTIST\_DISCIPLINE, EXHIBITION\_ARTWORK, ARTWORK\_TAG\_MAP, MEMBER\_FAVORITE\_DISCIPLINE.

**2FN** | Dans les tables à clé simple, tous les attributs non-clés dépendent entièrement de cette clé. Dans les tables d'association à clé composée (EXHIBITION\_ARTWORK, ARTIST\_DISCIPLINE, etc.), il n'existe aucun attribut non-clé susceptible de dépendre d'une seule partie de la clé.

**3FN** | Aucune dépendance transitive n'existe entre attributs non-clés. Les informations descriptives d'un artiste ne sont pas répétées dans ARTWORK ou WORKSHOP : on stocke seulement une clé étrangère. Les informations d'une galerie ne sont pas répétées dans EXHIBITION. Les disciplines et les tags sont dans des référentiels séparés. REVIEW et BOOKING sont des entités autonomes avec leurs propres clés primaires.

## Créer le schéma de la base de données

```
1 • CREATE DATABASE IF NOT EXISTS artconnect
2     CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
3 USE artconnect;
4
5 • CREATE TABLE artist (
6     artist_id INT AUTO_INCREMENT PRIMARY KEY,
7     name VARCHAR(100) NOT NULL,
8     bio TEXT,
9     birth_year INT,
10    contact_email VARCHAR(150) NOT NULL UNIQUE,
11    phone VARCHAR(30),
12    city VARCHAR(100),
13    website VARCHAR(255),
14    social_media VARCHAR(255),
15    is_active TINYINT(1) NOT NULL DEFAULT 1,
16    KEY idx_artist_name (name)
17 );
18
19 • CREATE TABLE gallery (
20     gallery_id INT AUTO_INCREMENT PRIMARY KEY,
21     name VARCHAR(150) NOT NULL,
22     address VARCHAR(255) NOT NULL,
23     owner_name VARCHAR(100),
24     opening_hours VARCHAR(100),
25     contact_phone VARCHAR(30),
26     rating DECIMAL(3,1),
27     website VARCHAR(255),
28     CONSTRAINT chk_gallery_rating CHECK (rating IS NULL OR (rating >= 0 AND rating <= 5))
29 );
30
31 • CREATE TABLE discipline (
32     discipline_id INT AUTO_INCREMENT PRIMARY KEY,
33     name VARCHAR(100) NOT NULL UNIQUE
34 );
35
36 • CREATE TABLE artwork_tag (
37     tag_id INT AUTO_INCREMENT PRIMARY KEY,
38     name VARCHAR(100) NOT NULL UNIQUE
39 );
40
41 • CREATE TABLE community_member (
42     member_id INT AUTO_INCREMENT PRIMARY KEY,
43     name VARCHAR(100) NOT NULL,
44     email VARCHAR(150) NOT NULL UNIQUE,
45     birth_year INT,
46     phone VARCHAR(30),
47     city VARCHAR(100),
48     membership_type VARCHAR(50)
49 );
50
51 • CREATE TABLE app_user (
52     user_id INT AUTO_INCREMENT PRIMARY KEY,
53     username VARCHAR(100) NOT NULL UNIQUE,
54     password VARCHAR(255) NOT NULL,
55     role ENUM('USER', 'ADMIN') NOT NULL DEFAULT 'USER',
56     member_id INT DEFAULT NULL,
57     CONSTRAINT fk_user_member FOREIGN KEY (member_id)
58         REFERENCES community_member(member_id) ON DELETE SET NULL
59 );
```

```

61 • CREATE TABLE artwork (
62     artwork_id INT AUTO_INCREMENT PRIMARY KEY,
63     title VARCHAR(150) NOT NULL,
64     creation_year INT,
65     type VARCHAR(100) NOT NULL,
66     medium VARCHAR(100),
67     dimensions VARCHAR(100),
68     description TEXT,
69     price DECIMAL(15,2),
70     status VARCHAR(20) NOT NULL,
71     artist_id INT NOT NULL,
72     CONSTRAINT fk_artwork_artist FOREIGN KEY (artist_id) REFERENCES artist(artist_id)
73     ON UPDATE CASCADE ON DELETE RESTRICT,
74     CONSTRAINT chk_artwork_price CHECK (price IS NULL OR price >= 0),
75     CONSTRAINT chk_artwork_status CHECK (status IN ('FOR_SALE','SOLD','EXHIBITED'))
76 );
77
78 • CREATE TABLE exhibition (
79     exhibition_id INT AUTO_INCREMENT PRIMARY KEY,
80     title VARCHAR(150) NOT NULL,
81     start_date DATE NOT NULL,
82     end_date DATE,
83     description TEXT,
84     curator_name VARCHAR(100),
85     theme VARCHAR(150),
86     gallery_id INT NOT NULL,
87     CONSTRAINT fk_exhibition_gallery FOREIGN KEY (gallery_id) REFERENCES gallery(gallery_id)
88     ON UPDATE CASCADE ON DELETE RESTRICT,
89     CONSTRAINT chk_exhibition_dates CHECK (end_date IS NULL OR end_date >= start_date)
90 );
91
92 • CREATE TABLE workshop (
93     workshop_id INT AUTO_INCREMENT PRIMARY KEY,
94     title VARCHAR(150) NOT NULL,
95     date DATETIME NOT NULL,
96     duration_minutes INT NOT NULL,
97     max_participants INT NOT NULL,
98     price DECIMAL(10,2),
99     instructor_artist_id INT NOT NULL,
100     location VARCHAR(150),
101     description TEXT,
102     level VARCHAR(30) NOT NULL,
103     CONSTRAINT fk_workshop_artist FOREIGN KEY (instructor_artist_id)
104     REFERENCES artist(artist_id) ON UPDATE CASCADE ON DELETE RESTRICT,
105     CONSTRAINT chk_workshop_price CHECK (price IS NULL OR price >= 0),
106     CONSTRAINT chk_workshop_duration CHECK (duration_minutes > 0),
107     CONSTRAINT chk_workshop_max_participants CHECK (max_participants > 0),
108     CONSTRAINT chk_workshop_level CHECK (level IN ('Beginner','Intermediate','Advanced'))
109 );
110
111 • CREATE TABLE workshop_registration (
112     registration_id INT AUTO_INCREMENT PRIMARY KEY,
113     member_id INT NOT NULL,
114     workshop_id INT NOT NULL,
115     registration_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
116     CONSTRAINT unique_registration UNIQUE (member_id, workshop_id),
117     CONSTRAINT fk_registration_member FOREIGN KEY (member_id)
118     REFERENCES community_member(member_id) ON DELETE CASCADE,
119     CONSTRAINT fk_registration_workshop FOREIGN KEY (workshop_id)
120     REFERENCES workshop(workshop_id) ON DELETE CASCADE
121 );

```

```

123 • CREATE TABLE review (
124     review_id          INT AUTO_INCREMENT PRIMARY KEY,
125     reviewer_member_id INT NOT NULL,
126     artwork_id         INT NOT NULL,
127     rating              INT NOT NULL,
128     comment             TEXT,
129     review_date         DATE NOT NULL,
130     CONSTRAINT fk_review_member FOREIGN KEY (reviewer_member_id)
131         REFERENCES community_member(member_id) ON UPDATE CASCADE ON DELETE CASCADE,
132     CONSTRAINT fk_review_artwork FOREIGN KEY (artwork_id)
133         REFERENCES artwork(artwork_id) ON UPDATE CASCADE ON DELETE CASCADE,
134     CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5)
135 );
136
137 • CREATE TABLE booking (
138     booking_id      INT AUTO_INCREMENT PRIMARY KEY,
139     workshop_id     INT NOT NULL,
140     member_id       INT NOT NULL,
141     booking_date    DATETIME NOT NULL,
142     payment_status  VARCHAR(30) NOT NULL,
143     CONSTRAINT fk_booking_workshop FOREIGN KEY (workshop_id)
144         REFERENCES workshop(workshop_id) ON UPDATE CASCADE ON DELETE CASCADE,
145     CONSTRAINT fk_booking_member FOREIGN KEY (member_id)
146         REFERENCES community_member(member_id) ON UPDATE CASCADE ON DELETE CASCADE,
147     CONSTRAINT chk_booking_status CHECK (payment_status IN ('PENDING','PAID','FAILED','CANCELLED'))
148 );
149
150 • CREATE TABLE exhibition_artwork (
151     exhibition_id INT NOT NULL,
152     artwork_id   INT NOT NULL,
153     PRIMARY KEY (exhibition_id, artwork_id),
154     CONSTRAINT fk_ea_exhibition FOREIGN KEY (exhibition_id)
155         REFERENCES exhibition(exhibition_id) ON UPDATE CASCADE ON DELETE CASCADE,
156     CONSTRAINT fk_ea_artwork FOREIGN KEY (artwork_id)
157         REFERENCES artwork(artwork_id) ON UPDATE CASCADE ON DELETE CASCADE
158 );
159
160 • CREATE TABLE artist_discipline (
161     artist_id      INT NOT NULL,
162     discipline_id  INT NOT NULL,
163     PRIMARY KEY (artist_id, discipline_id),
164     CONSTRAINT fk_ad_artist FOREIGN KEY (artist_id)
165         REFERENCES artist(artist_id) ON UPDATE CASCADE ON DELETE CASCADE,
166     CONSTRAINT fk_ad_discipline FOREIGN KEY (discipline_id)
167         REFERENCES discipline(discipline_id) ON UPDATE CASCADE ON DELETE CASCADE
168 );
169
170 • CREATE TABLE member_favorite_discipline (
171     member_id      INT NOT NULL,
172     discipline_id  INT NOT NULL,
173     PRIMARY KEY (member_id, discipline_id),
174     CONSTRAINT fk_mfd_member FOREIGN KEY (member_id)
175         REFERENCES community_member(member_id) ON UPDATE CASCADE ON DELETE CASCADE,
176     CONSTRAINT fk_mfd_discipline FOREIGN KEY (discipline_id)
177         REFERENCES discipline(discipline_id) ON UPDATE CASCADE ON DELETE CASCADE
178 );

```

```
180 • ○ CREATE TABLE artwork_tag_map (  
181     artwork_id INT NOT NULL,  
182     tag_id      INT NOT NULL,  
183     PRIMARY KEY (artwork_id, tag_id),  
184     CONSTRAINT fk_atm_artwork FOREIGN KEY (artwork_id)  
185         REFERENCES artwork(artwork_id) ON UPDATE CASCADE ON DELETE CASCADE,  
186     CONSTRAINT fk_atm_tag      FOREIGN KEY (tag_id)  
187         REFERENCES artwork_tag(tag_id) ON UPDATE CASCADE ON DELETE CASCADE  
188 );
```

## Étape 3 : Implémentation de la Base de Données et Fonctionnalités Avancées

### Création et remplissage de la base

#### **Prompt utilisé dans le LLM :**

Je travaille sur une base de données MySQL pour une plateforme artistique appelée ArtConnect. Le schéma contient les tables suivantes : artist, gallery, discipline, artwork\_tag, community\_member, artwork, exhibition, workshop, review, booking, exhibition\_artwork, artist\_discipline, member\_favorite\_discipline, artwork\_tag\_map. Génère un script SQL MySQL INSERT INTO cohérent avec ces contraintes : au moins 6 artistes, au moins 6 œuvres, au moins 4 galeries, au moins 4 expositions, au moins 4 workshops, au moins 6 membres, plusieurs associations artistes-disciplines, associations œuvres-tags, associations expositions-œuvres, plusieurs reviews et bookings, données réalistes et cohérentes, dates valides, pas de violation de clés étrangères, niveaux de workshop parmi Beginner/Intermediate/Advanced, status d'œuvre parmi FOR\_SALE/SOLD/EXHIBITED, payment\_status parmi PENDING/PAID/FAILED/CANCELLED. Retourne uniquement du SQL.

#### **Données présentes en base :**

La base contient les données d'exemple générées ainsi que des données ajoutées manuellement au cours des tests. Les artistes présents sont notamment Claude Monet, Ansel Adams, Frida Kahlo, Auguste Rodin, Sofia Turner et Batman (artiste de test ajouté manuellement). Les comptes utilisateurs (app\_user) incluent un compte admin avec le rôle ADMIN et plusieurs comptes USER (alice, mathieumdg, student, mathieu, robin, joker, alfred), tous avec des mots de passe hashés en BCrypt. Les inscriptions de workshops dans workshop\_registration reflètent des tests effectués depuis l'interface.

## Script d'insertion généré :

```
1 • USE artconnect;
2
3 • INSERT INTO discipline (name) VALUES
4   ('Painting'),
5   ('Sculpture'),
6   ('Photography'),
7   ('Digital Art'),
8   ('Mixed Media'),
9   ('Printmaking');
10
11 • INSERT INTO artwork_tag (name) VALUES
12   ('Portrait'),
13   ('Landscape'),
14   ('Modern'),
15   ('Classic'),
16   ('BlackAndWhite'),
17   ('Abstract');
18
19 • INSERT INTO artist (name, bio, birth_year, contact_email, phone, city, website, social_media, is_active) VALUES
20   ('Leonardo Vinci', 'Renaissance painter and workshop instructor.', 1452, 'leo@vincistudio.it', '+39-111-111', 'Florence', 'www.vincistudio.it', '@leo_vinci', TRUE),
21   ('Claude Monet', 'Master of impressionist painting.', 1840, 'claudemonet.fr', '+33-222-222', 'Giverny', 'www.monet.fr', '@claudemonet', TRUE),
22   ('Ansel Adams', 'Landscape and black-and-white photographer.', 1902, 'ansel@adams.co', '+1-333-333', 'San Francisco', 'www.anseladams.co', '@ansel_adams', TRUE),
23   ('Frida Kahlo', 'Iconic painter focused on identity and symbolism.', 1907, 'frida@kahlo.mx', '+52-444-444', 'Mexico City', 'www.fridakahlo.mx', '@frida_kahlo', TRUE),
24   ('Auguste Rodin', 'French sculptor and art mentor.', 1840, 'auguste@rodin.fr', '+33-555-555', 'Paris', 'www.rodin.fr', '@auguste_rodin', TRUE),
25   ('Sofia Turner', 'Digital and mixed-media contemporary artist.', 1992, 'sofia@turner.art', '+44-666-666', 'London', 'www.sofiaturner.art', '@sofia_turner', TRUE);
26
```



```

27 • INSERT INTO gallery (name, address, owner_name, opening_hours, contact_phone, rating, website) VALUES
28 ('Louvre Art House', 'Rue de Rivoli, Paris', 'Marie Laurent', '09:00-18:00', '+33-701-111', 4.9, 'www.louvrearthouse.fr'),
29 ('The British Gallery', 'Great Russell St, London', 'James Carter', '10:00-18:30', '+44-702-222', 4.7, 'www.britishgallery.uk'),
30 ('Metropolitan Hub', '1000 5th Ave, New York', 'Olivia Stone', '09:30-19:00', '+1-703-333', 4.8, 'www.metropolitanhub.us'),
31 ('Modern Arts Center', '22 Art Street, Berlin', 'Hans Keller', '10:00-20:00', '+49-704-444', 4.6, 'www.modernartscenter.de');
32
33 • INSERT INTO community_member (name, email, birth_year, phone, city, membership_type) VALUES
34 ('Alice Wonderland', 'alice@art.com', 1998, '+33-801-001', 'Paris', 'Premium'),
35 ('Bob Ross', 'bob@happytrees.com', 1985, '+44-801-002', 'London', 'Standard'),
36 ('Charlie Brown', 'charlie@peanuts.com', 1995, '+1-801-003', 'New York', 'Student'),
37 ('Diana Moore', 'diana@culture.org', 1990, '+49-801-004', 'Berlin', 'Premium'),
38 ('Emma Davis', 'emma@creative.net', 2001, '+33-801-005', 'Lyon', 'Standard'),
39 ('Farid Khan', 'farid@artsocial.io', 1993, '+212-801-006', 'Casablanca', 'Premium');
40
41 • INSERT INTO artist_discipline (artist_id, discipline_id) VALUES
42 (1, 1),
43 (2, 1),
44 (3, 3),
45 (4, 1),
46 (5, 2),
47 (6, 4),
48 (6, 5);
49
50 • INSERT INTO member_favorite_discipline (member_id, discipline_id) VALUES
51 (1, 1),
52 (1, 2),
53 (2, 1),
54 (3, 3),
55 (4, 4),
56 (5, 5),
57 (6, 2),
58 (6, 4);
59
60 • INSERT INTO artwork (title, creation_year, type, medium, dimensions, description, price, status, artist_id) VALUES
61 ('Mona Lisa', 1503, 'Painting', 'Oil on wood', '77x53 cm', 'Portrait of Lisa Gherardini.', 850000000.00, 'FOR_SALE', 1),
62 ('The Last Supper', 1498, 'Painting', 'Tempera on gesso', '460x880 cm', 'Religious mural masterpiece.', 450000000.00, 'EXHIBITED', 1),
63 ('Water Lilies', 1916, 'Painting', 'Oil on canvas', '200x180 cm', 'Series inspired by Monet's garden.', 40000000.00, 'FOR_SALE', 2),
64 ('The Two Fridas', 1939, 'Painting', 'Oil on canvas', '173x173 cm', 'Double self-portrait.', 5000000.00, 'EXHIBITED', 4),
65 ('The Thinker', 1904, 'Sculpture', 'Bronze', '186x98x145 cm', 'Famous bronze sculpture.', 15000000.00, 'FOR_SALE', 5),
66 ('Monolith, The Face of Half Dome', 1927, 'Photography', 'Gelatin silver print', '40x50 cm', 'Iconic black-and-white landscape.', 100000.00, 'SOLD', 3),
67 ('Neon Fragments', 2022, 'Digital Art', 'Digital mixed media', '4K', 'Contemporary digital composition.', 12000.00, 'FOR_SALE', 6),
68 ('Urban Pulse', 2023, 'Mixed Media', 'Acrylic and digital print', '120x90 cm', 'City-inspired hybrid artwork.', 18000.00, 'FOR_SALE', 6);
69
70 • INSERT INTO artwork_tag_map (artwork_id, tag_id) VALUES
71 (1, 1),
72 (1, 4),
73 (2, 4),
74 (3, 2),
75 (4, 1),
76 (4, 6),
77 (5, 3),
78 (6, 2),
79 (6, 5),
80 (7, 3),
81 (7, 6),
82 (8, 3);
83
84 • INSERT INTO exhibition (title, start_date, end_date, description, curator_name, theme, gallery_id) VALUES
85 ('Renaissance Revival', '2026-03-09', '2026-04-15', 'A celebration of classical Renaissance masterpieces.', 'Marie Laurent', 'Classic Renaissance', 1),
86 ('Sculpting the Soul', '2026-03-25', '2026-05-01', 'Exploration of sculptural expression.', 'James Carter', 'Modern & Classical Sculpture', 2),
87 ('Impressionist Dreams', '2026-02-09', '2026-03-20', 'Light, color and poetic atmosphere.', 'Olivia Stone', 'Light and Color', 3),
88 ('Digital Horizons', '2026-04-05', '2026-05-30', 'Contemporary digital and hybrid artworks.', 'Hans Keller', 'Future of Expression', 4);
89

```

```

90 • INSERT INTO exhibition_artwork (exhibition_id, artwork_id) VALUES
91 (1, 1),
92 (1, 2),
93 (2, 5),
94 (3, 3),
95 (3, 4),
96 (4, 7),
97 (4, 8),
98 (4, 6);
99
100 • INSERT INTO workshop (title, date, duration_minutes, max_participants, price, instructor_artist_id, location, description, level) VALUES
101 ('Mastering Oil Painting', '2026-04-14 10:00:00', 180, 15, 150.00, 1, 'Louvre Art House', 'Hands-on introduction to oil painting methods.', 'Intermediate'),
102 ('Impressionist Landscapes', '2026-04-19 14:00:00', 150, 20, 120.00, 2, 'Metropolitan Hub', 'Painting landscapes with light and movement.', 'Beginner'),
103 ('Sculpting Modernity', '2026-04-24 09:30:00', 210, 12, 200.00, 5, 'The British Gallery', 'Study of expressive sculpture techniques.', 'Advanced'),
104 ('Creative Digital Layers', '2026-05-03 11:00:00', 160, 18, 90.00, 6, 'Modern Arts Center', 'Experimenting with digital and mixed-media workflows.', 'Intermediate');
105
106 • INSERT INTO review (reviewer_member_id, artwork_id, rating, comment, review_date) VALUES
107 (1, 1, 5, 'A timeless masterpiece.', '2026-03-10'),
108 (2, 3, 4, 'Beautiful colors and atmosphere.', '2026-03-11'),
109 (3, 6, 5, 'Powerful photograph with great depth.', '2026-03-12'),
110 (4, 7, 4, 'Very modern and engaging.', '2026-04-06'),
111 (5, 5, 5, 'A monumental sculpture.', '2026-03-28'),
112 (6, 4, 4, 'Emotional and striking composition.', '2026-03-13');
113
114 • INSERT INTO booking (workshop_id, member_id, booking_date, payment_status) VALUES
115 (1, 1, '2026-04-01 09:00:00', 'PAID'),
116 (1, 2, '2026-04-01 10:00:00', 'PENDING'),
117 (2, 3, '2026-04-02 11:15:00', 'PAID'),
118 (2, 5, '2026-04-02 12:00:00', 'PAID'),
119 (3, 4, '2026-04-03 14:20:00', 'PAID'),
120 (4, 6, '2026-04-04 16:00:00', 'PENDING'),
121 (4, 1, '2026-04-04 16:30:00', 'PAID');

```

## Vues, index et droits d'accès

### Vue 1 : vw\_artwork\_details

Objectif : simplifier les requêtes de lecture des œuvres depuis l'interface en évitant une jointure systématique côté application. Expose le titre, type, medium, prix, statut, et les informations de l'artiste associé.

```

1 • CREATE OR REPLACE VIEW vw_artwork_details AS
2 SELECT
3     a.artwork_id,
4     a.title,
5     a.type,
6     a.medium,
7     a.price,
8     a.status,
9     ar.name AS artist_name,
10    ar.city AS artist_city
11 FROM artwork a
12 JOIN artist ar ON a.artist_id = ar.artist_id;

```

## Vue 2 : vw\_workshop\_booking\_summary

Objectif : offrir une vision consolidée du remplissage de chaque workshop (nombre total de réservations et nombre de réservations payées) sans que l'application ait à effectuer des agrégations SQL complexes.

```
1 • CREATE OR REPLACE VIEW vw_workshop_booking_summary AS
2 SELECT
3     w.workshop_id,
4     w.title,
5     w.date,
6     w.max_participants,
7     COUNT(b.booking_id) AS total_bookings,
8     SUM(CASE WHEN b.payment_status = 'PAID' THEN 1 ELSE 0 END) AS paid_bookings
9 FROM workshop w
10 LEFT JOIN booking b ON w.workshop_id = b.workshop_id
11 GROUP BY w.workshop_id, w.title, w.date, w.max_participants;
```

## Vue 3 : vw\_exhibition\_catalog

Objectif : présenter un catalogue complet des expositions en une seule requête, masquant la complexité des quatre jointures nécessaires.

```
1 • CREATE OR REPLACE VIEW vw_exhibition_catalog AS
2 SELECT
3     e.exhibition_id,
4     e.title      AS exhibition_title,
5     e.theme,
6     g.name       AS gallery_name,
7     a.title      AS artwork_title,
8     ar.name      AS artist_name
9 FROM exhibition e
10 JOIN gallery g      ON e.gallery_id = g.gallery_id
11 JOIN exhibition_artwork ea ON e.exhibition_id = ea.exhibition_id
12 JOIN artwork a      ON ea.artwork_id = a.artwork_id
13 JOIN artist ar      ON a.artist_id = ar.artist_id;
```

## Index créés

```
1  -- Optimise la recherche par nom dans l'onglet Artists
2 • CREATE INDEX idx_artist_name ON artist(name);
3
4  -- Optimise les recherches par titre dans l'onglet Artworks
5 • CREATE INDEX idx_artwork_title ON artwork(title);
6
7  -- Index couvrant les requêtes de comptage de places (utilisé par le trigger de capacité et la vue vw_workshop_booking_summary)
8 • CREATE INDEX idx_booking_workshop_status ON booking(workshop_id, payment_status);
```

L'index idx\_artist\_name est directement utile pour la fonctionnalité de recherche par nom dans l'onglet Artists. L'index composite sur booking couvre à la fois la condition WHERE workshop\_id = ? du trigger de capacité et le GROUP BY + SUM de la vue de résumé, évitant un full scan de la table à chaque requête.

## Déclencheurs et programmes stockés

### Trigger 1 : trg\_workshop\_check\_capacity\_before\_insert

Avant chaque insertion dans booking, vérifie que le nombre de réservations existantes pour ce workshop n'atteint pas déjà le maximum autorisé. Protège l'intégrité métier indépendamment de la couche applicative

```
1  DELIMITER $$
2 • CREATE TRIGGER trg_workshop_check_capacity_before_insert
3  BEFORE INSERT ON booking
4  FOR EACH ROW
5  BEGIN
6      DECLARE v_count INT;
7      DECLARE v_max   INT;
8      SELECT COUNT(*) INTO v_count FROM booking WHERE workshop_id = NEW.workshop_id;
9      SELECT max_participants INTO v_max FROM workshop WHERE workshop_id = NEW.workshop_id;
10     IF v_count >= v_max THEN
11         SIGNAL SQLSTATE '45000'
12         SET MESSAGE_TEXT = 'Booking refused: workshop is full.';
13     END IF;
14 END$$
15 DELIMITER ;
```

## Trigger 2 : trg\_review\_check\_rating\_before\_insert

Avant chaque insertion dans review, contrôle que la note est bien entre 1 et 5. Constitue une double protection avec la contrainte CHECK déjà présente sur la table.

```
1  DELIMITER $$
2  ● CREATE TRIGGER trg_review_check_rating_before_insert
3  BEFORE INSERT ON review
4  FOR EACH ROW
5  BEGIN
6  IF NEW.rating < 1 OR NEW.rating > 5 THEN
7      SIGNAL SQLSTATE '45000'
8      SET MESSAGE_TEXT = 'Rating must be between 1 and 5.';
9  END IF;
10 END$$
11 DELIMITER ;
```

## Trigger 3 : trg\_exhibition\_check\_dates\_before\_insert

Avant chaque insertion dans exhibition, vérifie que la date de fin est postérieure ou égale à la date de début.

```
1  DELIMITER $$
2  ● CREATE TRIGGER trg_exhibition_check_dates_before_insert
3  BEFORE INSERT ON exhibition
4  FOR EACH ROW
5  BEGIN
6  IF NEW.end_date IS NOT NULL AND NEW.end_date < NEW.start_date THEN
7      SIGNAL SQLSTATE '45000'
8      SET MESSAGE_TEXT = 'Exhibition end_date must be >= start_date.';
9  END IF;
10 END$$
11 DELIMITER ;
```

## Procédure sp\_add\_booking

Encapsule la création d'une réservation de workshop avec son statut de paiement.

```
1      DELIMITER $$
2  ● ○ CREATE PROCEDURE sp_add_booking(
3      IN p_workshop_id    INT,
4      IN p_member_id      INT,
5      IN p_payment_status VARCHAR(30)
6  )
7  ○ BEGIN
8      INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
9      VALUES (p_workshop_id, p_member_id, NOW(), p_payment_status);
10  END$$
11  DELIMITER ;
```

## Procédure sp\_create\_exhibition

Encapsule la création complète d'une exposition avec tous ses attributs.

```
1      DELIMITER $$
2  ● ○ CREATE PROCEDURE sp_create_exhibition(
3      IN p_title           VARCHAR(150),
4      IN p_start_date     DATE,
5      IN p_end_date       DATE,
6      IN p_description     TEXT,
7      IN p_curator_name   VARCHAR(100),
8      IN p_theme           VARCHAR(150),
9      IN p_gallery_id     INT
10  )
11  ○ BEGIN
12      INSERT INTO exhibition (title, start_date, end_date, description, curator_name, theme, gallery_id)
13      VALUES (p_title, p_start_date, p_end_date, p_description, p_curator_name, p_theme, p_gallery_id);
14  END$$
15  DELIMITER ;
```

## Fonction fn\_count\_paid\_bookings

Retourne le nombre de réservations effectivement payées pour un workshop donné. Utile pour calculer le taux de remplissage réel (payé) plutôt que nominal.

```
1  DELIMITER $$
2  • CREATE FUNCTION fn_count_paid_bookings(p_workshop_id INT)
3  RETURNS INT
4  DETERMINISTIC
5  BEGIN
6      DECLARE v_total INT;
7      SELECT COUNT(*) INTO v_total
8      FROM booking
9      WHERE workshop_id = p_workshop_id AND payment_status = 'PAID';
10     RETURN v_total;
11 END$$
12 DELIMITER ;
```

## Script de test des objets SQL

```
1  -- Test des trois vues
2  • SELECT * FROM vw_artwork_details;
3  • SELECT * FROM vw_workshop_booking_summary;
4  • SELECT * FROM vw_exhibition_catalog;
5
6  -- Test des procédures
7  • CALL sp_add_booking(2, 6, 'PAID');
8  • CALL sp_create_exhibition(
9      'Photography and Light', '2026-06-10', '2026-07-05',
10     'A curated exhibition around photography and light.',
11     'Olivia Stone', 'Visual Atmospheres', 3
12 );
13
14 -- Test de la fonction
15 • SELECT fn_count_paid_bookings(2) AS paid_bookings_for_workshop_2;
--
```

## Transactions

### Transaction validée — inscription simultanée à deux workshops :

```
1 • START TRANSACTION;
2
3 • INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
4 VALUES (1, 3, NOW(), 'PAID');
5
6 • INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
7 VALUES (4, 3, NOW(), 'PAID');
8
9 • COMMIT;
```

Les deux insertions sont atomiques : si l'une échoue le ROLLBACK implicite annule l'ensemble, évitant qu'un membre se retrouve inscrit à un seul workshop sur deux.

### Transaction annulée — référence inexistante :

```
1 • START TRANSACTION;
2
3 • INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
4 VALUES (3, 2, NOW(), 'PAID');
5
6 -- workshop_id 999 n'existe pas → violation de clé étrangère
7 • INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
8 VALUES (999, 2, NOW(), 'PAID');
9
10 • ROLLBACK;
```

La transgression de clé étrangère sur la seconde insertion déclenche un rollback complet : la première réservation n'est pas non plus appliquée, garantissant la cohérence de la base.

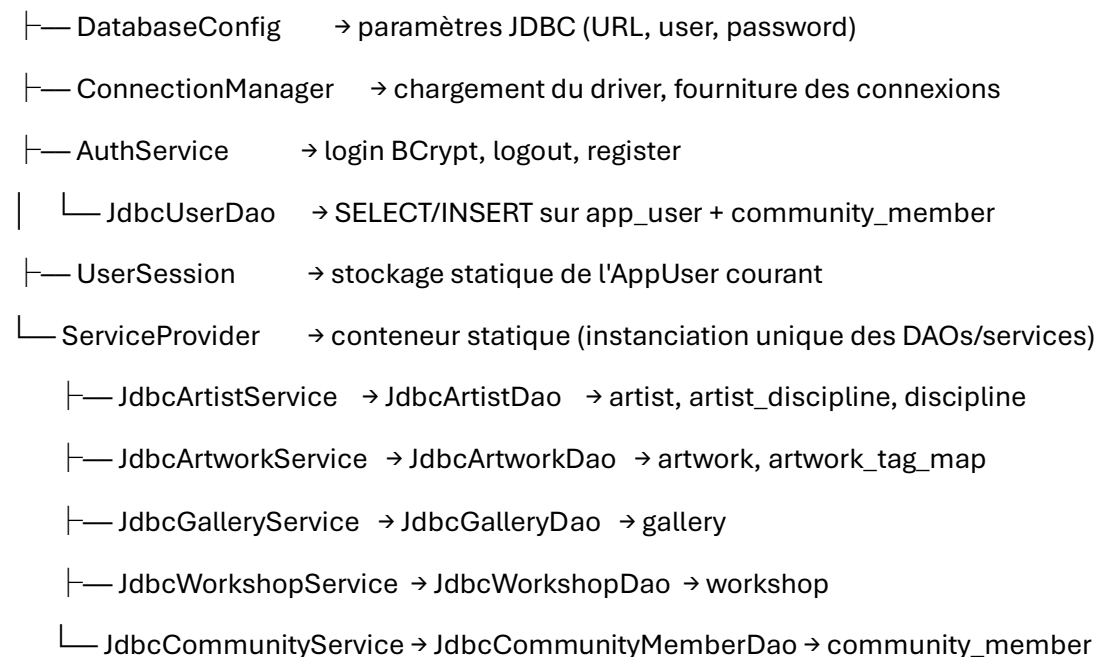


# Étape 4 : Intégration de la base dans l'application Java ArtConnect

## Architecture de la couche de persistance

L'application suit une architecture en couches. Elle est orchestrée dès le démarrage par MainApp :

MainApp



WorkshopController → WorkshopRegistrationService → INSERT dans workshop\_registration

AdminController → connexion directe via DatabaseConfig (CRUD toutes entités)

## Connexion à la base — DatabaseConfig et ConnectionManager

DatabaseConfig résout les paramètres de connexion selon une priorité : propriété système Java (-Dartconnect.db.url) > variable d'environnement (ARTCONNECT\_DB\_URL) > valeur par défaut (jdbc:mysql://localhost:3306/artconnect?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true). Le même mécanisme s'applique à l'utilisateur (root par défaut) et au mot de passe (vide par défaut). Cette approche permet de déployer l'application dans différents environnements sans recompilation.

## Implémentation des DAO JDBC

Chaque DAO implémente l'interface correspondante du package dao et utilise systématiquement des `PreparedStatement` pour toutes les opérations, protégeant contre les injections SQL. Les ressources (connexion, statement, resultset) sont fermées via try-with-resources.

**JdbcArtistDao** : `findAll()` charge tous les artistes puis, pour chacun, exécute un second `PreparedStatement` pour charger leurs disciplines via la jointure `artist_discipline` → `discipline`. `save()`, `update()` et `delete()` opèrent par le nom de l'artiste comme identifiant fonctionnel dans l'UI. `findByCity()` filtre sur la colonne `city`.

**JdbcArtworkDao** : gère les œuvres avec leur statut (`Artwork.Status` enum Java mappé sur le `VARCHAR` SQL) et leur référence à l'artiste.

**JdbcUserDao** : implémente `findByUsername()` (retourne un `Optional<AppUser>`) et `register()`. L'inscription crée d'abord un enregistrement dans `community_member`, récupère l'ID généré, puis insère dans `app_user` avec le mot de passe hashé en BCrypt.

## Services

`ServiceProvider` instancie une seule fois chaque DAO et service au chargement statique de la classe. Tous les controllers accèdent aux services via `ServiceProvider.getService()`, garantissant une instance unique par service pendant toute la durée de vie de l'application.

`AuthService` gère le cycle d'authentification. `login()` récupère l'`AppUser` via `JdbcUserDao.findByUsername()` puis vérifie le mot de passe avec `BCrypt.checkpw()`. En cas de succès, il stocke l'utilisateur dans `UserSession`. `logout()` vide la session. `register()` délègue à `JdbcUserDao.register()` et encapsule les exceptions.

`WorkshopRegistrationService` est un service dédié à la fonctionnalité d'inscription depuis l'UI. Il récupère le `member_id` de l'utilisateur courant via `UserSession.getCurrentUser().getMemberId()` et exécute un `INSERT` dans `workshop_registration`. La contrainte `UNIQUE` (`member_id`, `workshop_id`) de la table gère les doublons au niveau SQL ; toute exception est capturée et retourne `false`, ce qui déclenche un message d'erreur dans le `WorkshopController`.

## Adaptation de l'interface graphique

**MainApp** : ouvre `LoginView` au lancement. Si l'utilisateur ferme la fenêtre sans se connecter, l'application s'arrête (`System.exit(0)`). En cas de connexion réussie, charge `MainView.fxml` et injecte l'`AuthService` dans `MainController` pour gérer la déconnexion et l'affichage conditionnel du panneau Admin.

**WorkshopController** : alimente un TableView (colonnes titre, date, instructeur, prix, niveau) via JdbcWorkshopService. Un Timeline JavaFX rafraîchit automatiquement la liste toutes les 5 secondes. Le bouton Register appelle WorkshopRegistrationService.registerToWorkshop() avec l'ID du workshop sélectionné et affiche une alerte selon le résultat.

**AdminController** : panneau complet avec six ListView (une par entité) et six formulaires de saisie correspondants. Chaque entité dispose des actions Ajouter, Modifier et Supprimer connectées directement à la base via PreparedStatement. Les erreurs de suppression sont interceptées et traduites : code MySQL 1451 (violation FK) donne un message indiquant quelle relation bloque la suppression et quoi supprimer d'abord ; code 1062 (doublon) indique un email ou nom déjà existant. Les ComboBox (artiste instructeur, galerie d'exposition, statut d'œuvre, niveau de workshop) sont alimentées dynamiquement depuis la base.